

PHPは何を捨て、 どんな力を手に入れてきたのか

What did **PHP** give up and what power did it get?

！ お前誰よ



- うさみけんた (@tadsan) / Zonu.EXE
- GitHub/Packagistでは id: zonuexe
- ピクシブ株式会社 pixiv事業本部
- Emacs Lisper, PHPer
 - Emacs PHP Modeのメンテナ引き継ぎました
 - 好きなリスプはEmacs Lispです
- Qiitaに記事を書いたり変なコメントしてるよ

pixiv



お絵描きがもっと楽しくなる場所を創る

pixiv 2018年度新卒
エントリー受付中



Friends of Emacs-PHP development

Join us!

 <?php

-  **Repositories** 25
-  People 5
-  Teams 1
-  Projects 0
-  Settings

Find a repository...

Type: All ▾

Language: All ▾

Customize pinned repositories

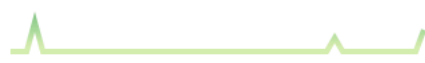
New

php-runtime.el

PHP-Emacs bridge, call PHP function from Emacs

- php
- emacs
- melpa

 Emacs Lisp ★ 2  1  GPL-3.0 Updated 2 days ago



phpactor.el

Interface to Phpactor (an intelligent code-completion and refactoring tool for PHP)

 Emacs Lisp ★ 8  4 1 issue needs help Updated 3 days ago



php-mode

A PHP mode for GNU Emacs



Top languages

 Emacs Lisp  PHP

Most used topics

Manage

emacs

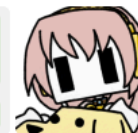
melpa

php

major-mode

People

5 >



Invite someone



Search or jump to...



[Pull requests](#)

[Issues](#)

[Marketplace](#)

[Explore](#)



emacs-jp

📍 Japan

🔗 <http://emacs-jp.github.com/>

Repositories 18

People 37

Teams 8

Projects 0

Settings

Find a repository...

Type: All ▾

Language: All ▾

[Customize pinned repositories](#)

New

reading-init.el

init.el読書会のページ

[html](#)

[dotfiles](#)

[emacs](#)

[emacs-lisp](#)

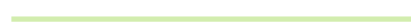
● Ruby ★ 6 🍴 1 Updated 6 days ago



helm-c-yasnippet

Helm source for yasnippet

● Emacs Lisp ★ 14 🍴 5 Updated on 21 Mar



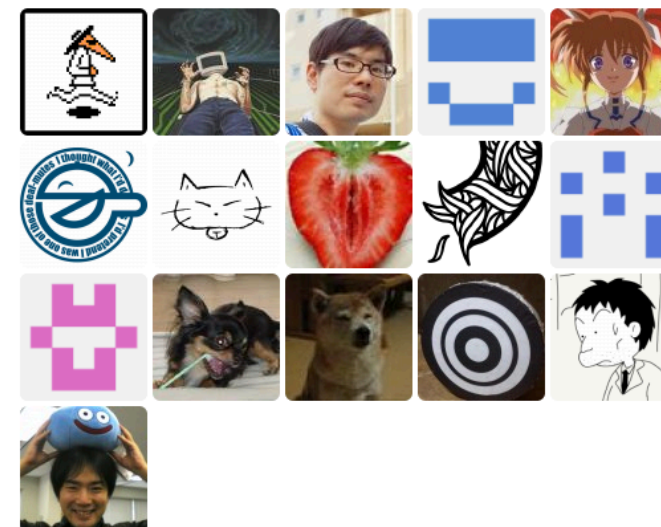
replace-colorthemes

Top languages

● Emacs Lisp ● Ruby ● TeX

People

37 >



これまでのあらすじ

昨日



11月
30

大改修！PHPレガシーコードビフォーアフター



5人の匠が問題設計大改修！

主催：PHPerKaigi / @tomzoh

\$PHP_legacy_code ->
before()->after();

グループ

メンバーです

PHPerKaigi



イベント数 **1回**
メンバー数 **216人**

開催前

2019/11/30(土)

13:00 ~ 18:40

Googleカレンダー icsファイル

このイベントに申し込む

開催日時が重複しているイベントに申し込んでいる場合、このイベントには申し込むことができません

募集期間

2019/11/12(火) 08:28 ~
2019/11/30(土) 18:40

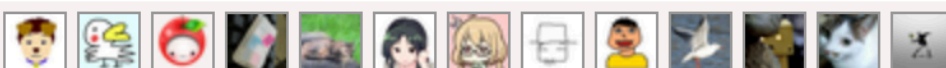
イベントへのお問い合わせ

人気

人気



フォロー参加者



フォローブックマーク



募集内容

一般
無料

先着順
159/180人

運営お手伝い
無料

先着順
6/8人

PHPプロジェクトに 静的解析を新規導入する

Introduce static analyzers to PHP project



大改修！PHPレガシーコードビフォーアフター
2019-11-30 #phperkaigi

そもそもPHP自体が
レガシーと言われがち

PHPカンファレンス 2016

pixiv inside [archive]



2016-11-11

PHP

Phan静的解析がもたらす大PHP型検査時代

こんにちは、pixivでPHPをやってるうさみです。健全なコードベースは黙っても降ってこないの
で、チーム全体で開発効率を高めるような改善をするのがお仕事です。

テキストエディタはmicro推しです ヲ(//><)ノ ☆

さる11月3日に大田区産業プラザ PiOで開催されたPHPカンファレンス 2016にて大怪獣に蹂躪さ
れながらPhanについて30分のセッション発表をいたしましたので、その内容を紹介します！

本日のお題

[ホーム](#) / [PHP Conference Japan 2019](#) / [トーク](#)

/ PHPは何を捨て、どんな力を手に入れてきたのか by うさみけんた

採択 2019/12/01 14:10～ Track 5 (1F AB会議室 - サテライト: 6F C/E会議室) 25分枠

PHPは何を捨て、どんな力を手に入れてきたのか PHP Conference Japan 2019



うさみけんた



[tadsan](#)

PHPの言語仕様は変化の歴史であり、常にほかのプログラミング言語の特徴を取り入れることで発展してきました。その変化はときに断絶を生んできましたが、同時に我々に多くの可能性をもたらしてきました、このトークでは一人のプログラミング言語好きとしてPHPの言語仕様の歴史的経緯と強力な機能を紹介していきます。

☆ 18

そもそもPHPというプログラミング言語はどんな機能を持っているのか

*“We have things like protected properties.
We have abstract methods. We have all
this stuff that your computer science
teacher told you you should be using.
I don't care about this crap at all.”*

–Rasmus Lerdorf

https://en.wikiquote.org/wiki/Rasmus_Lerdorf

“PHPには*protected*プロパティも抽象メソッドもありますよ。計算機科学の教授が「使え」と言ってるものは全部。そんなことは~~クソ~~興味ないですけど”

—Rasmus Lerdorf

僕が思いつくPHPの
機能をざっと挙げて
みると...

抽象クラス, トレイト, インターフェイス, ジェネレータ, イテレータ, クロージャ, 関数の動的呼び出し, 無名クラス, eval, 型宣言, 連想配列, リフレクションAPI, 自己文書化, 標準入出力の読み書き, REPL, クラスの遅延ロード

改めてPHPの言語
仕様が追加された時
系列を見てみよう

| PHP 7.4

- Arrow functions 2.0
 - JavaScriptなど
- Typed Property
 - Javaなどの静的型付き言語
- Numeric Literal Separator
 - Ada, Ruby, JavaScriptなど

| PHP 7.3

- Flexible Heredoc and Nowdoc
 - Rubyにも似た記法はある
- list() Reference Assignment
 - これはPHP独自に近い

| PHP 7.2

- Trailing Commas In List Syntax
 - カンマ(,)を並べて書く記法のほぼ全てで余分につけてもよくなった
- Parameter Type Widening
 - 継承したクラスで引数の型省略

| PHP 7.1

- Nullable Types
- Void Return Type
- Multi catch
- Allow specifying keys in list()

| PHP 7.0

- Anonymous Class
- Generator Delegation(yield from)
- Return Type Declaration
- Null Coalesce Operator
- Scalar type hints

| PHP 5.6

- Exponential Operator(pow**)
- Importing named function
- Variadic function argument (...)
 - 配列を引数として展開
- Argument unpacking(...)
 - 可変長引数を配列として取得

| PHP 5.5

- Generator
- finally keyword (try-catch-...)
- Const array/string dereference
- Allow non-scalar keys in foreach

| PHP 5.4

- Traits
- Short array syntax
- callable type hint

| PHP 5.3

- Closures (function expression)
 - 関数式、クロージャ、またはラムダ式
 - JavaScriptのfunctionに似てるが、レキシカルスコープ(変数の捕捉)のためにuseキーワードが必要

ひとつひとつの機能
の話を読んだら20分
では全然足りない

なので本日はPHPがどう変わっていったかというエピソードを交えてPHPの言語機能を紹介していきます

さて

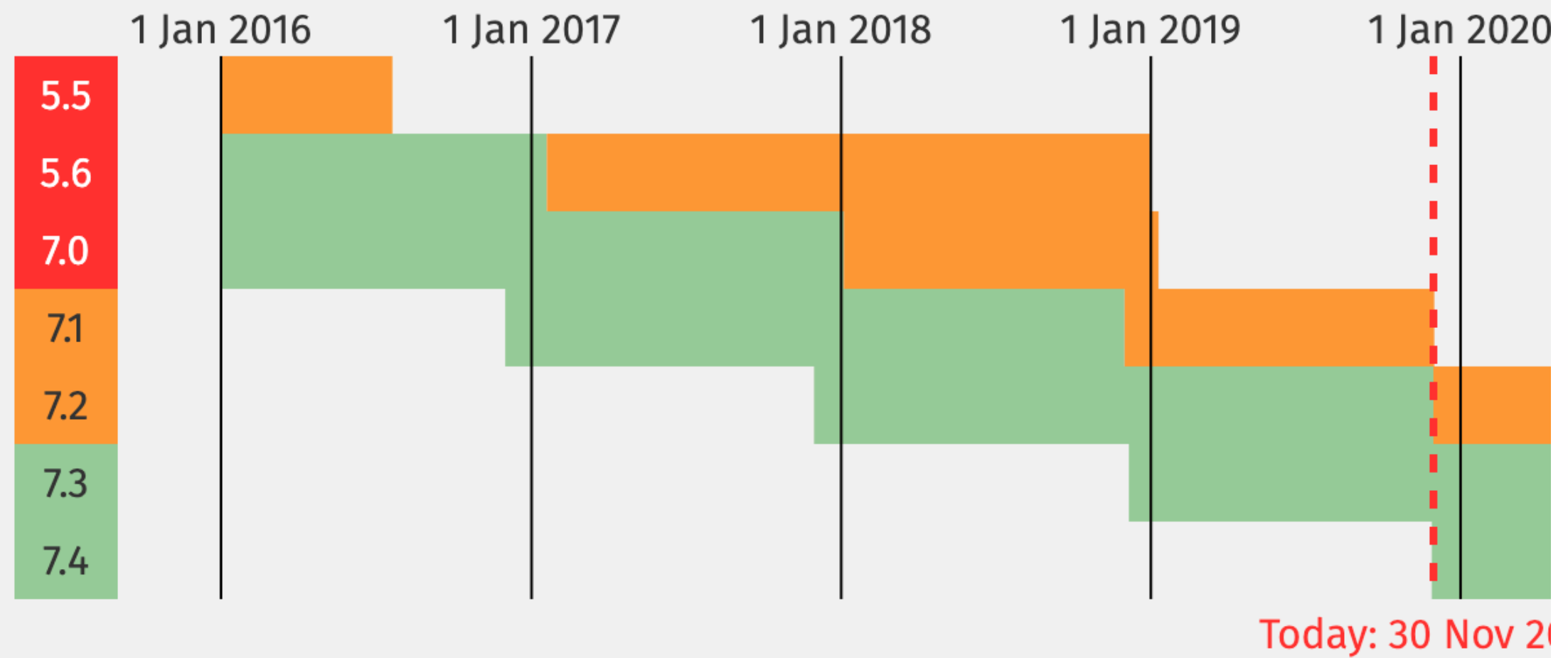
PHP 7.4.0がリリース
されましたね

本日12月1日は
何の日でしょう？

Currently Supported Versions

Branch	Initial Release		Active Support Until		Security Support Until	
<u>7.1</u>	1 Dec 2016	<i>2 years, 11 months ago</i>	1 Dec 2018	<i>11 months ago</i>	1 Dec 2019	<i>midnight</i>
<u>7.2</u>	30 Nov 2017	<i>2 years ago</i>	30 Nov 2019	<i>midnight</i>	30 Nov 2020	<i>in 11 months</i>
<u>7.3</u>	6 Dec 2018	<i>11 months ago</i>	6 Dec 2020	<i>in 1 year</i>	6 Dec 2021	<i>in 2 years</i>
<u>7.4</u>	28 Nov 2019	<i>2 days ago</i>	28 Nov 2021	<i>in 1 year, 11 months</i>	28 Nov 2022	<i>in 2 years, 11 months</i>

Or, visualised as a calendar:



27

この数は何でしょう

Request for Comments

This page gives an overview of the current RFCs for PHP.

To create a new RFC, see [How To Create an RFC](#).

Note: An RFC is effectively “owned” by the person that created it. If you want to make changes, get permission from the creator. If no agreement can be found, the only course of action is to create a competing RFC. In this case, the old RFC page should be modified to become an intermediate page that points to all the competing RFC's.

A new page in this RFC namespace will automatically be loaded with an RFC [template](#). Customize as needed.



せいはい

PHP7.4に向けて
受理されたRFCの数

RFC?

| RFC(Request for Comment)

- 新バージョンへの変更提案
- PHPの機能の改廃の提案はRFCを通じて行われる
- 提案はメーリングリストで議論されPHP.net VCSアカウントの所持者が議決する

| PHP RFCの歴史

- 2008年頃から機能提案のために運用されはじめた
- 2011年(5.4alpha1)から正式なリリースプロセスになった

“PHP releases have always been done spontaneously, in a somehow chaotic way. Individual(s) decided when a release will happen and what could or could fit in. Release managers role are unclear and the way to nominate them is not clearly defined either.”

– Request for Comments: Release Process
<https://wiki.php.net/rfc/releaseprocess>

“PHPのリリースは常に無秩序な方法で自発的に行われてきました。個々人がいつリリースされるか、何が起こるか、そして何が収まるかを決定しています。リリースマネージャの役割は不明瞭で、指名の手続きも明確に定義されていません。”

– Request for Comments: Release Process
<https://wiki.php.net/rfc/releaseprocess>

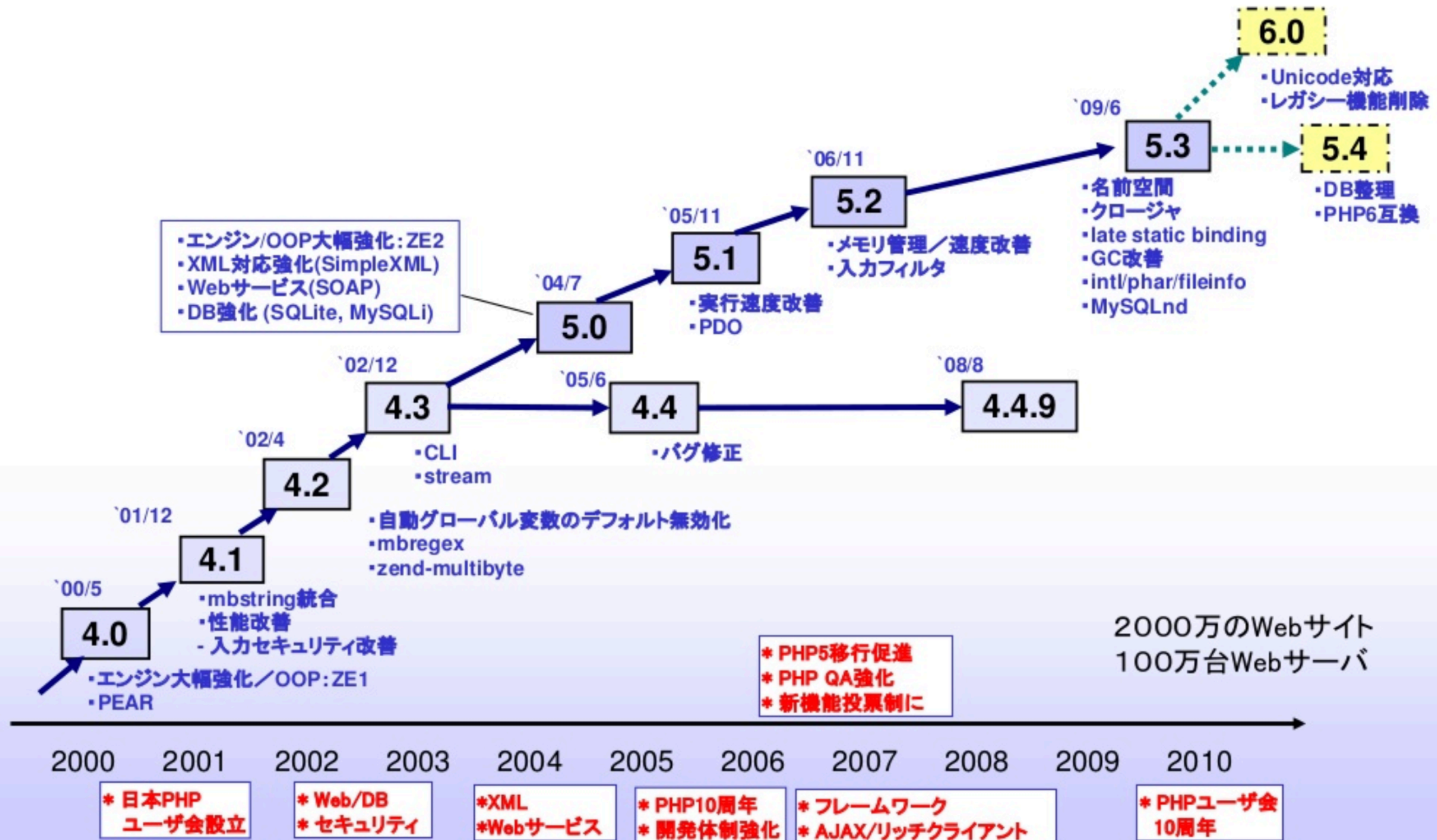
この時期(2008~2010年)
のPHPには
何があったでしょう

この当時のPHP情勢

廣川さんの
「PHPの今とこれから」が
当時の事情の貴重な資料



PHPの歩み



– PHPの今とこれから2009

<https://www.slideshare.net/hirokawa/php2009>



PHP 6.0

- PHP 6は遅延中： 2009/5に開発者会議開催
- 国際化／Unicode化
 - Unicodeネイティブ対応(ICU): 常にオン
 - 国際化機能: intlエクステンション (PHP 5.3と共通化)
 - 書き直しの範囲大: パーサ(re2c)、DB拡張(PDO)など
 - DB: 接続文字コード、テーブル文字コード、カラム文字コードの区別
- レガシーコード削除
 - register_globals, magic_*, safe_mode
- 論議中:
 - 識別子で大文字／小文字区別 PHP 6.1 (PHP 5.4で警告)
 - Traits 継承の柔軟化

– PHPの今とこれから2009

<https://www.slideshare.net/hirokawa/php2009>



PHP 5.3 (1/2)

- 2009年6月リリース
- PHP6までのつなぎ(=当面の本命):
PHP6 - (Unicode) + (pecl/intl)
- レガシーコード整理: ZE1互換モード廃止
- ガーベッジコレクタ改良: 循環コレクタ
- Late Static Binding: スタティック宣言を実行時に解決
- Dynamic Static Call: 動的変数によりスタティックコール
- 名前空間: PHP 6からバックポート
- ラムダ関数/クロージャ

– PHPの今とこれから2009

<https://www.slideshare.net/hirokawa/php2009>

Unicode化とは
どういうことか？

| 言語処理系の文字列実装

- CSI方式(Code Set Independent)
- 特定のコードセットに依存しない
- UCS正規化方式(Universal Character Set)
- 内部的に文字コードを統一してから処理する
- https://magazine.rubyist.net/articles/0025/0025-Ruby19_m17n.html

| 言語ごとの文字列実装

- CSI方式
 - Ruby, Python 2, PHP
- UCS正規化方式
 - Java, JavaScript, Perl, Python 3, PHP6

ざっくりCSI方式

- 文字列変換をしないので、オーバーヘッドは最小限
- サポートする文字コードごとに文字列処理の実装が要る

ざっくりUCS正規化方式

- 文字列を変換するので入出力で変換オーバーヘッドがある
- 文字列処理は内部文字コード用のものだけ用意すればいい

| PHP6はUTF-16を目指した

- UCS正規化方式への移行
- 従来のPHP文字列はバイト列
- PHP 5.2で(binary)キャストと b"string" が追加された
- 結局Unicode化しなかった
ので、ただのエイリアス

| 重要なこと

- UCS正規化しないとUnicodeサポートできないわけではない
- UTF-16にしても1コードポイント=1文字になるわけではない (サロゲートペア)

| PHP6.0の顛末

- 実装は難航し、文字列以外の6.0向け機能は5.3, 5.4系でそれぞれリリースされた
- 2010年3月にリポジトリのtrunk(master)が5.3にロールバックした

PHP 6

From:	Rasmus Lerdorf	Date:	Thu, 11 Mar 2010 17:22:48 +0000
Subject:	PHP 6		
Groups:	php.internals		

Ah, Jani went a little crazy today in his typical style to force a decision. The real decision is not whether to have a version 5.4 or not, it is all about solving the Unicode problem. The current effort has obviously stalled. We need to figure out how to get development back on track in a way that people can get on board. We knew the Unicode effort was hugely ambitious the way we approached it. There are other ways.

So I think Lukas and others are right, let's move the PHP 6 trunk to a branch since we are still going to need a bunch of code from it and move development to trunk and start exploring lighter and more approachable ways to attack Unicode. We have a few already. Enhanced mbstring and ext/intl. Let's see some good ideas around that and work on those in trunk. Other features necessarily need to play along with these in the same branch. I refuse to go down the path of a 5.4 branch and a separate Unicode branch again.

The main focus here needs to be to get everyone working in the same branch.

-Rasmus

開発が頓挫したPHP6は
欠番としてスキップされ
PHP5.6の次は7.0になっ
たというのは有名な話

Request for Comments: Release Process

- Version: 0.6
- Date: 2010-11-22
- Author: Felipe Pena, Etienne Kneuss, Stanislav Malyshev, Gustavo André dos Santos Lopes, David Soria Parra, Christian Stocker, Rob Richards, Pierre Joye, Zeev Suraski, Ilia Alshanetsky
- Status: Accepted, [Voting results](#)

Introduction

PHP releases have always been done spontaneously, in a somehow chaotic way. Individual(s) decided when a release will happen and what could or could fit in. Release managers role are unclear and the way to nominate them is not clearly defined either.

(私はこの時代にPHPに
触れておらず議論を詳し
くは追ってないので
想像の域は出ないが)

この時代にPHPはカウボーイ
による開発スタイルを捨て、明
文化されたスケジュールとプロ
セスによる開発体制に移行した

PHPのリリリース計画や
メンテナンス期間が定め
られたのもこのRFCから

PHP5.3ではPHP6
を待たず重要な
機能がマージされた



PHP 5.3 (1/2)

- 2009年6月リリース
- PHP6までのつなぎ(=当面の本命):
PHP6 - (Unicode) + (pecl/intl)
- レガシーコード整理: ZE1互換モード廃止
- ガーベッジコレクタ改良: 循環コレクタ
- Late Static Binding: スタティック宣言を実行時に解決
- Dynamic Static Call: 動的変数によりスタティックコール
- 名前空間: PHP 6からバックポート
- ラムダ関数/クロージャ

– PHPの今とこれから2009

<https://www.slideshare.net/hirokiawa/php2009>

| 遅延静的束縛

- `static::method();`
- 親クラスの静的メソッドから継承されたクラスのメソッドを呼び出す
- ちなみに`forward_static_call()`関数のマニュアルには「静的メソッドをコールする」と書いてるが、この説明は間違い（翻訳の問題ではない）

| Dynamic static call

- Hoge::fuga()の代わりに
\$class="Hoge"; \$class::fuga();
で呼べるようになった
- 静的メソッド版可変関数

| 名前空間

- クラスや関数を階層化できる
 - 名前空間違いの同名のクラスが作れるようになった
- フルネームをFQSENと呼ぶ
- useでファイルローカルな別名を付けることもできる

| useの誤解されがちなところ

- include/requireのようにファイルを読み込む機能ではない
- ファイル内で呼べるようローカルな名前を付けるだけ
- 存在しないクラスをuseしてもエラーにはならない

| クロージャ (関数式・ラムダ式)

- ローカルな関数が作れる
- ただの関数を作れるだけではなく、外側の変数を引き込めるのが重要 (関数閉包)
- 普通の関数やメソッドにその機能はない

| クロージャ以前の無名関数

- `create_function()`という関数で関数式っぽいことはできた
- その実態は`eval`で `lambda_1`のような関数を動的に定義する
- この関数はリスキーなので PHP7.2からdeprecated

| PHPのクロージャ

- 構文上はJavaScriptのものに似ているが、`use`で列挙しないと外側の変数を捕捉できない
- この仕様にはPHPの変数の説明しにくい挙動が関わっている…

| PHPの変数

- PHPで変数の値を再代入したり引数に渡したりすると、
(意味論として)値がコピーされる
- クロージャでuseしたときも
PHPでは引数と同じ動きになる
- &を付けると変数の参照をとる

| JavaScriptのクロージャ

```
make_counter = function() {  
    i = 0;  
    return function () {  
        return i++;  
    };  
};
```

```
c1 = make_counter();
```

```
console.log(c1());  
console.log(c1());  
console.log(c1());
```

| PHPのクロージヤ

この記述だと\$用=0のコピーされるだけ。&を付けると変数リファレンスで値を更新できる

```
<?php
$make_counter = function() {
    $i = 0;
    return function () use ($i) {
        return $i++;
    };
};

$c1 = $make_counter();

var_dump($c1()); //=> 0
var_dump($c1()); //=> 0
var_dump($c1()); //=> 0
```

| 最初のRFCクロージャ

- 実はJavaScript風クロージャよりも前に短いクロージャが提案されてた (2008年3月)
 - RFC: Short syntax for anonymous functions
 - ただしDraft止まり

Request for Comments: Short syntax for anonymous functions

- Version: 1.0
- Date: 2008-03-06
- Author: Marcello Duarte [✉ marcello.duarte@gmail.com](mailto:marcello.duarte@gmail.com)
- Status: Draft
- First Published at: [🌐 https://wiki.php.net/rfc/short-syntax-for-anonymous-functions](https://wiki.php.net/rfc/short-syntax-for-anonymous-functions)
- Other formats ..

– PHP: rfc:short-syntax-for-anonymous-functions
<https://wiki.php.net/rfc/short-syntax-for-anonymous-functions>

Syntax

An anonymous function in php could be expressed by a typical statement block, surrounded by curly brackets.

```
<?php
$sayHi = { echo "hi"; };
$sayHi(); // prints: hi

$sayHello = { $name => echo "hello, $name"; };
$sayHello("Chuck Norris"); // prints: hello, Chuck Norris

$sayHello = { $name, $mood => echo "hello, $name. It's $mood day!"; };
$sayHello("Chuck Norris", "wonderful"); // prints: hello, Chuck Norris.
```

– PHP: rfc:short-syntax-for-anonymous-functions
<https://wiki.php.net/rfc/short-syntax-for-anonymous-functions>

時は流れて
2019年

PHP RFC: Arrow Functions 2.0

- Date: 2019-03-12
- Author: Nikita Popov ✉ nikic@php.net
- Author: Levi Morrison ✉ levim@php.net
- Author: Bob Weinand ✉ bwoebi@php.net
- Target version: PHP 7.4
- Implementation: <https://github.com/php/php-src/pull/3941>
- Status: Accepted

PHP: rfc:arrow_functions_v2
https://wiki.php.net/rfc/arrow_functions_v2

Function signatures

The arrow function syntax allows arbitrary function signatures, including parameter and return types, default values, variadics, as well as by-reference passing and returning. All of the following are valid examples of arrow functions:

```
fn(array $x) => $x;  
fn(): int => $x;  
fn($x = 42) => $x;  
fn(&$x) => $x;  
fn&($x) => $x;  
fn($x, ...$rest) => $rest;
```

姿形は違うが幾度もの
議論の末に短い記法の
関数式が導入された

ところで

PHP RFC: Arrow Functions 2.0

- Date: 2019-03-12
- Author: Nikita Popov ✉ nikic@php.net
- Author: Levi Morrison ✉ levim@php.net
- Author: Bob Weinand ✉ bwoebi@php.net
- Target version: PHP 7.4
- Implementation: <https://github.com/php/php-src/pull/3941>
- Status: Accepted



PHP: rfc:arrow_functions_v2
https://wiki.php.net/rfc/arrow_functions_v2

私たちはこの
提案者の名前を
見たことがある！



Generator

(<https://wiki.php.net/rfc/generators>)

- Generator: イテレータを関数で定義(Python)
- 例: ファイルの各行を処理: ファイル全体読み込み→メモリが大量に必要
- イテレータ定義: 多くのコード記述が必要
- 配列全体を返すのではなく、値を生成するイテレータを返す

```
<?php
function xrange($start, $end, $step = 1) {
    for ($i = $start; $i < $end; $i += $step) {
        yield $i;
    }
}

foreach (xrange(10, 20) as $i) {
    echo $i;
}
```

Nikita Popov
17歳、ベルリン

– PHPの今とこれから2012

<https://www.slideshare.net/hirokawa/php-now-and-then-2012-at-php-conference-2012-tokyo-japan>

| ジェネレータ

- (古典的な) コルーチン
- 「何度も繰り返す」 構造を関数(メソッド)を使って簡単に作れる

| Nikita Popov

- 近年のPHPの中核的な開発者
- PHP5.5以降、非常に多数の言語機能を実装してPHPに取り入れてきた
- PHP-Parserなどの作者
- 去年の大学院を卒業し今年JetBrainsに入社してPhpStormチームに参加

とてもざっくり言うとしガ
シー仕様を消し、不条理な
挙動は是正し、他言語にあ
るようなカッコイイ機能を
追加提案してきた

不条理な
挙動って？

| 例: 未定義定数はその文字列になる

- echo PHP;
- => 文字列 "PHP" が出力
- E_NOTICE/E_WARNING
- error_reportingが低い環境
でさりげなく生き残ってた可
能性が高い

歴史的経緯で残ったレ
ガシー仕様はさくっと
消しまくっていいこうぜ

いいじゃん
いいじゃん







燃えた

◀ [RFC] [VOTE] Deprecate PHP's short open tags, again

✈ Posted [21 days ago](#) by **G. P. B.** — [view source](#)



The voting for the "Deprecate short open tags, again" [1] RFC has begun. It is expected to last two (2) weeks until 2019-08-20.

A counter argument to this RFC is available at https://wiki.php.net/rfc/counterargument/deprecate_php_short_tags

Best regards

George P. Banyard

[1] https://wiki.php.net/rfc/deprecate_php_short_tags_v2

✈ Posted [21 days ago](#) by **Chase Peeler** — [view source](#)



I'm not a voter, but, I have a question. If this fails, does that mean the original RFC that passed is still in effect?

If I did have a vote, I would be against this RFC for the reasons laid out by Zeev in the counter argument. However, I feel the negative impacts of possible code leaks that will be caused by the original RFC are even worse. If I did have a vote, I might feel forced to vote for this. At least with this RFC we wouldn't really see any of the other negative impacts until PHP 9, given us time to revisit and possibly reverse that part of this RFC.

The voting for the "Deprecate short open tags, again" [1] RFC has begun. It is expected to last two (2) weeks until 2019-08-20.

A counter argument to this RFC is available at https://wiki.php.net/rfc/counterargument/deprecate_php_short_tags

Best regards

George P. Banyard

[1] https://wiki.php.net/rfc/deprecate_php_short_tags_v2

| Deprecate PHP's short open tags, again

- 一般的なPHPタグは<?php ... ?> または<?= ... ?> だが、<? ... ?>という形式もある
- php.iniで有効無効を切替られる
- 環境によってOnだったりOffだったり

| Deprecate PHP's short open tags, again

- XML宣言との相性がとても悪い
- そんな半端モンは引導を渡してやろっぜ！ ということで提案された
- 構文がシンプルになるし、機械的に変換もできるから良いだろ、という主旨

Zeevによる 反論

Counterargument to Deprecate Short Tags RFC V2

- Date: 2019-08-06
- Author: Zeev Suraski, zeev@php.net

Introduction

An proposal has been made to deprecate PHP's short tags syntax. The proposal was first made available in [V1 here](#), and a newer version was later made available in [V2 here](#). The goal of this page is to provide a counterargument for this proposal, or in other words, to explain why we should not be deprecating this syntax.

Overview

The motivation for removing short tags as it is illustrated in the V2 RFC is quite weak. Essentially - it does not present *any* new motivations that did not exist 20+ years ago, when this syntax was first introduced. At the time, all the different potential opening and closing tags were heavily discussed, and all the implications - including the ones brought up by the deprecation RFC - were already known. There is no new 'evidence' available here.

– Table of Contents

Counterargument to Deprecate Short Tags RFC V2
Introduction
Overview
Analyzing The Motivations For Deprecation
The Case For Keeping Short Tags
Additional Thoughts
Summary

| Zeev Suraski

- PHP3・4の開発者
 - 相方のAndi Gutmansとともに
VM型処理系のZend Engine開発
- PHPでよくあるZend…は
ZeevとAndiに由来
- PHPコードに未だに影響を残す

さらなる提案

◀ Bringing Peace to the Galaxy

✈ Posted [19 days ago](#) by **Zeev Suraski** — [view source](#)



[... and not in the Sith Lord kind of way.]

Looking at some of the recent (& not so recent) discussions on internals@, some of the recent proposals, as well as some of the statements made regarding the future direction of the language - makes it fairly clear that we have a growing sense of polarization.

As Peter put it yesterday (I may be paraphrasing a bit) - some folks just want to clear some legacy stuff. I think that in practice it goes well beyond that - many on internals@ see parts of PHP as in bad need of repair (scoop: I agree with some of that), while other capabilities, that exist in other competing languages - are - in their opinion - sorely missing.

At the other end of the spectrum, we have folks who think that we should retain the strong bias for downwards compatibility we always had, that PHP isn't in dire need of an overhauling repair and that as far as features go

- less is more - and we don't have to race to replicate features from other languages - but rather opt for keeping PHP simple.

To a large degree, these views are diametrically opposed. This made many internals@ discussions turn into literally zero sum games - where when one side 'wins', the other side 'loses', and vice versa.

続きは
Webで

P++とは何だったのか

Bring peace to the galaxy with P++

[2019-08-28]

第141回 PHP勉強会@東京

#phpstudy

まとめ

PHPは力を手に入れ
ようとして失敗して
きた経験が結構ある

現在のPHPは静的解析
できる環境が年々強まっ
て往年のざっくりさは息
をひそめつつある

銀河の平和は
どっちだ